Chapter 23 : Probabilistic Language Processing

1. What is Probabilistic Language Processing?

- Natural language is inherently ambiguous and uncertain.
- A sentence can have multiple possible interpretations (meanings, structures, intents).
- Probabilistic language processing uses probability distributions to rank alternative interpretations based on data.
- Helps AI systems handle ambiguity, noise, and variation in real-world language.

Key Idea: Use probability to select the most likely interpretation from many possibilities rather than a single fixed answer.

2. Why Probability for Language?

- **Ambiguity:** many possible meanings (e.g., "bank" = river bank or financial bank).
- **Noise:** errors in speech recognition, typos, informal text.
- **Variation:** different ways to express the same meaning (e.g., "I'm hungry" vs "I want food").
- **Uncertainty:** limited context and incomplete information.
- **Data-driven ranking:** probability allows us to prefer more likely interpretations based on large corpora.

Probability turns linguistic uncertainty into a principled decision process.

3. Language Models

- A language model assigns a probability to a sequence of words.
- Denoted as $P(w_1, w_2, \ldots, w_n)$.
- Using the chain rule of probability:

$$P(w_1, \ldots, w_n) = \prod_{i=1}^{n} P(w_i \mid w_1, \ldots, w_{i-1})$$

- Gives the probability of each word conditioned on the previous words.
- Used in many NLP tasks:
  - Speech recognition (word sequences)
  - Machine translation (fluent output)
  - Predictive text (next word prediction)

4. n-gram Models

- Approximate the full history using only the last (n-1) words (Markov assumption).
- Estimate probabilities from large text corpora.
- Simple, effective, and widely used.

Model	Probability	Example
Unigram (n = 1)	$P(w_i)$	P("dog") (no context)
Bigram (n = 2)	$P(w_i \mid w_{i-1})$	P("barks" | "dog") (one previous word)
Trigram (n = 3)	$P(w_i \mid w_{i-2}, w_{i-1})$	P("loudly" | "dog","barks") (two previous words)

5. Smoothing and Sparsity

- **Problem:** Many possible word sequences never occur in the training data → zero counts (probability = 0).
- This is called data sparsity.
- Smoothing assigns small non-zero probability to unseen events.
- Common techniques:

Technique	Idea	Notes
Add-one (Laplace)	Add 1 to all counts.	Simple but can over-smooth.
Backoff	If n-gram unseen, use shorter context.	e.g., back off from trigram to bigram.
Interpolation	Combine multiple n-gram orders (e.g., unigram + bigram).	Weighted sum of different models.

6. Entropy & Perplexity

- Entropy measures the uncertainty in a probability distribution over the next word.
- Lower entropy means the model is more certain.
- Perplexity measures how well a language model predicts a test set. Lower perplexity is better.

Perplexity (for a sequence):

$$PP(W) = 2^{-\frac{1}{N}\sum_{i=1}^{N} \log_2 P(w_i \mid \text{context})}$$

- Intuitively: perplexity is the effective number of choices the model is confused among at each step.
- Used to compare different language models.

7. Mini Flow: Text Data → Model → Probability Estimates → Ranked Interpretations

Text Data (corpus) →
- Large collection of text
- Learn word statistics
- Capture real language usage

Train Language Model (e.g., n-gram) →
- Estimate n-gram probabilities
- Apply smoothing
- Build a probabilistic model from data

Probability Estimates $P(w_1, \ldots, w_n)$ →
- Compute probability of candidate sequences
- Use chain rule and n-gram model

Ranked Interpretations (most likely first)
- Rank alternatives by probability
- Choose the most likely interpretation
- Used in ASR, MT, parsing, etc.

8. Applications

Application	How Probability Helps	Example
Speech recognition	Rank possible word sequences	Most likely transcript
Spelling correction	Choose most probable correction	"hte" → "the"
Predictive text	Predict next word	"I would like to ..."
Parsing	Select most likely syntactic structure	Disambiguate sentences
Machine translation	Generate most fluent translation	Rank candidate translations

9. Mini Takeaway

Probability allows AI to handle the inherent uncertainty and ambiguity of human language by ranking alternative interpretations using data.

10. Key Takeaways

1. Natural language is ambiguous and uncertain.
2. Probabilistic models rank alternative interpretations.
3. Language models assign probabilities to word sequences.
4. n-gram models use local context and are estimated from data.
5. Smoothing handles unseen events and data sparsity.
6. Entropy and perplexity measure uncertainty and model quality.
7. Widely used in speech recognition, spelling correction, predictive text, parsing, and machine translation.

Chapter 23 : Probabilistic Language Processing

1. Hidden Markov Models (HMMs)

- HMM is a probabilistic model for sequences with hidden states that generate observable outputs.
- Widely used in language processing, speech recognition, and bioinformatics.
- Assumes first-order Markov property.
- Parameters learned from data.
- Captures sequential structure and uncertainty in language.

Key Idea: A sequence of hidden linguistic states (e.g., parts of speech) generates the observed words with certain probabilities.

2. Hidden States and Observations

- Hidden states (S): unobserved variables (e.g., POS tags such as Noun, Verb, Adj).
- Observations (O): observed words in the sentence.
- At each time step t:
  - a hidden state s_t is active.
  - it emits an observation word w_t.
- The state sequence is hidden, but we observe the word sequence.
- Goal: infer the most likely state sequence given the observations.

Notation:

States: $S = \{s_1, s_2, \ldots, s_N\}$

Observations: $O = \{w_1, w_2, \ldots, w_T\}$

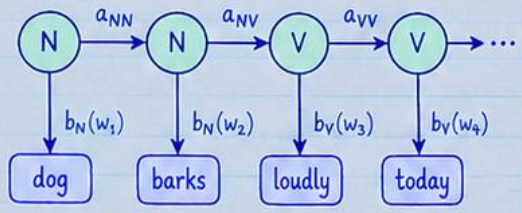
3. HMM Structure and Probabilities

- Transition probabilities:
  $a_{ij} = P(s_t = s_j \mid s_{t-1} = s_i)$
  (probability of moving from state i to j)
- Emission probabilities:
  $b_j(w) = P(w_t = w \mid s_t = s_j)$
  (probability of generating word w from state j)
- Initial state distribution:
  $\pi_i = P(s_1 = s_i)$
- Joint probability of a state sequence S and observation sequence O:

$$P(S, O) = \pi_{s_1} \prod_{t=2}^{T} a_{s_{t-1} s_t} \prod_{t=1}^{T} b_{s_t}(w_t)$$

4. Hidden Markov Model Example

HMM for a simple sentence with two states: Noun (N) and Verb (V).



Each hidden state emits an observed word with an emission probability, and transitions to the next state with a transition probability.

5. Part-of-Speech (POS) Tagging

- POS tagging assigns a part-of-speech tag to each word in a sentence.
- HMM formulation:
  - Hidden states: POS tags (e.g., Noun, Verb, Adjective, etc.)
  - Observations: words in the sentence.
- Goal: find the most likely sequence of tags given the words.

Example:

Sentence: The dog barks loudly

Most likely tags: Det | Noun | Verb | Adv

6. Viterbi Algorithm

- Finds the most likely sequence of hidden states (tags) given the observations.
- Uses dynamic programming.
- Maintains the highest probability path to each state at each time step.
- Recurrence relation:

$$\delta_t(j) = \max_i \left[\delta_{t-1}(i) a_{ij} \right] b_j(w_t)$$

- Backpointer to recover the best path.
- Complexity: $O(TN^2)$ for T words and N states.

7. Forward Algorithm

- Computes the probability of the observation sequence P(O) by summing over all possible state sequences.
- Uses dynamic programming (forward messages).
- Recurrence relation:

$$\alpha_t(j) = \left[\sum_i \alpha_{t-1}(i) a_{ij} \right] b_j(w_t)$$

- Initialization: $\alpha_1(j) = \pi_j \, b_j(w_1)$
- Final probability: $P(O) = \sum_j \alpha_T(j)$
- Complexity: $O(TN^2)$.
- Useful for evaluating how likely a sentence is under the model.

8. Viterbi vs. Forward Algorithm

Aspect	Viterbi Algorithm	Forward Algorithm
Goal	Most likely state sequence (arg max S)	Total probability of observations P(O)
Computation	Maximization over paths	Summation over all paths
Recurrence	$\delta_t(j) = \max_i \left[\delta_{t-1}(i) a_{ij} \right] b_j(w_t)$	$\alpha_t(j) = \sum_i \alpha_{t-1}(i) a_{ij} \, b_j(w_t)$
Output	Best state sequence	Probability P(O)
Uses	Sequence labeling (e.g., POS tagging)	Model evaluation, parameter learning (e.g., Baum-Welch)
Complexity	$O(TN^2)$	$O(TN^2)$

9. Sequence Labeling in NLP

- Many NLP tasks can be cast as sequence labeling.
- Examples: part-of-speech tagging, named entity recognition (NER), chunking, shallow parsing, word segmentation.
- HMM provides a simple but powerful framework.
- Additional features (e.g., richer state sets, higher-order models, or discriminative models) can improve performance.

10. Key Takeaways

1. HMM models sequences with hidden states and observable words.
2. Transition and emission probabilities define the model.
3. POS tagging is a classic application in NLP.
4. Viterbi finds the most likely tag sequence using dynamic programming.
5. Forward algorithm computes the probability of the observation sequence.
6. Sequence labeling is widely used in many language processing tasks.
7. HMMs provide a simple, effective probabilistic model for structured language data.

Chapter 23 : Probabilistic Language Processing

1. Probabilistic Context-Free Grammars (PCFGs)

- A PCFG assigns a probability to each production rule in a context-free grammar (CFG).
- Each set of rules with the same left-hand side must sum to 1.
- Generates a probability distribution over complete parse trees, and thus over sentences.
- Often learned from treebanks using relative frequencies.

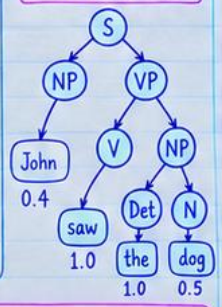
Key Idea: A PCFG captures the uncertainty in syntactic structure by assigning probabilities to grammar rules.

2. PCFG Example

- Grammar rules (subset):

S → NP VP	1.0
NP → Det N	0.6
NP → 'John'	0.4
VP → V NP	0.7
VP → V	0.3
Det → 'the'	1.0
N → 'dog'	0.5
N → 'cat'	0.5
V → 'saw'	1.0

Parse tree for "John saw the dog"



Probability of this parse tree:
$0.4 \times 1.0 \times 0.7 \times 1.0 \times 1.0 \times 0.5 = 0.14$

3. Grammar vs. Probabilistic Grammar

Aspect	CFG (Grammar)	PCFG
Rule form	$A \to \alpha$	$A \to \alpha$ [p]
Information	Only allows/disallows structures	Assigns probability to each rule
Parse output	Set of all possible parses	Probability distribution over parses
Disambiguation	Does not prefer any parse	Prefers more probable parses
Learning	Usually hand-written	Learned from tagged treebanks

4. Probabilistic Parsing

- Goal: find the most probable parse tree for a sentence using a PCFG.
- Most probable parse:

$$T^* = \arg\max_{T \in \text{Trees}(w)} P(T \mid w)$$

- $P(T \mid w)$ is proportional to the product of rule probabilities used in T (assuming independence).
- Can also compute the total probability of a sentence by summing over all possible parses.
- Unlike CFG parsing, we prefer the most likely syntactic interpretation.

5. Chart Parsing (Dynamic Programming)

- Direct enumeration of all parse trees is expensive due to many overlapping subproblems.
- Chart parsing stores and reuses partial results using dynamic programming.
- For PCFGs, algorithms like the probabilistic CYK (for CNF grammars) or probabilistic Earley compute the most probable parse efficiently.
- Each chart entry stores the best probability for a constituent spanning words $i..j$.
- Combines smaller constituents to build larger ones.

Key Idea: Use a chart to avoid recomputing subtrees and to efficiently find the most probable parse (often in $O(n^3)$ time).

Example: Chart entries for "the dog saw a cat"

	0	1	2	3	4	5
0		Det (0.6)				S (0.12)
1			N (0.5)		V (1.0)	
2				Det (0.6)		
3					N (0.5)	
4						

Each cell stores the best probability for a constituent spanning words $i..j$.

6. Lexicalized Parsing

- Standard PCFGs condition rules only on nonterminals, which can be too coarse.
- Lexicalized parsing conditions on head words, e.g., NP(w) → Det N(w) or VP(V) → V NP.
- Head words help capture subcategorization preferences and long-distance dependencies.
- Lexicalized PCFGs (e.g., Collins model) significantly improve parsing accuracy.

Key Idea: Use lexical information (head words and subcategorization) to prefer more realistic syntactic structures.

7. Semantic Disambiguation

- Syntactic parsing may still produce multiple plausible interpretations with different meanings.
- Use semantic information to select the most plausible interpretation.
- Sources of semantic knowledge:
  - Selectional preferences (e.g., "eat" prefers food).
  - Semantic roles and thematic relations.
  - World knowledge and ontologies (e.g., WordNet).
- Can be modeled probabilistically, e.g.,

$$P(\text{meaning} \mid \text{sentence}) \propto P(\text{sentence} \mid \text{meaning}) \times P(\text{meaning})$$

Example: "I saw the man with the telescope"
Prefer the interpretation where the man has the telescope (more plausible in context).

8. Noisy-Channel Model

- View language understanding as inference in a noisy communication channel.
- A speaker intends a meaning (or target sentence) X, and the observed sentence w is a noisy version of X.
- Choose the most probable intended X:

$$X^* = \arg\max_X P(w \mid X)\, P(X)$$

- $P(w \mid X)$: channel model (probability of observing w given X, e.g., speech errors, spelling errors).
- $P(X)$: prior probability from a language model (e.g., PCFG, n-gram).
- Used in speech recognition, spelling correction, machine translation, and general noisy text understanding.

9. Applications and Examples

Task	How PCFGs / Probabilistic Models Help	Example
Syntactic parsing	Choose most probable parse tree.	"I saw the man with the telescope"
Machine translation	Model source/target structure probabilistically.	Translate with highest probability.
Speech recognition	Noisy-channel decoding with language model.	Map audio to most probable word sequence.
Spelling correction	Noisy channel: correct to most probable intended word.	"teh" → "the"
Information extraction	Use parses to find relations with higher confidence.	Find subject-verb-object.

10. Key Takeaways

1. PCFGs assign probabilities to grammar rules and define a distribution over parse trees.
2. Probabilistic parsing finds the most likely syntax tree.
3. Chart parsing uses dynamic programming to compute probable parses efficiently.
4. Lexicalized parsing uses head words to improve accuracy.
5. Semantic disambiguation uses world knowledge and selectional preferences to choose meanings.
6. The noisy-channel model chooses the most probable intended sentence given the observed one.
7. Probabilistic language processing is widely used in parsing, speech recognition, machine translation, and spelling correction.

Chapter 23 : Probabilistic Language Processing

1. Speech Recognition

- Goal: find the most likely word sequence given an observed acoustic signal.
- Use Bayes rule to combine acoustic model with a language model.
- Acoustic model $P(\text{signal} \mid W)$: probability of the sound given the words.
- Language model $P(W)$: prior probability of the word sequence.

Best word sequence:

$$W^* = \arg\max_W \underbrace{P(\text{signal} \mid W)}_{\text{acoustic model}} \underbrace{P(W)}_{\text{language model}}$$

2. Components of a Speech Recognizer

- Acoustic model: relates sounds to phones/words (e.g., HMM).
- Pronunciation model: maps words to sequences of phones.
- Language model: probability of word sequences (e.g., n-gram model).
- Decoding: search for the most likely word sequence (e.g., Viterbi search).
- Trained on large collections of transcribed speech.

Combines uncertain acoustic evidence with linguistic context to recognize spoken words.

3. Machine Translation (MT)

- Goal: find the most likely translation in the target language.
- Statistical MT uses parallel corpora (aligned sentence pairs).
- Model the probability of target sentence T given source sentence S.

Best translation:

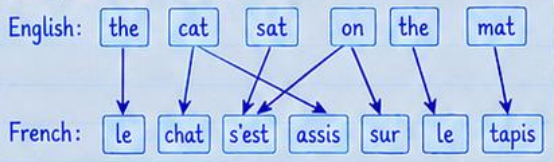
$$T^* = \arg\max_T P(T \mid S)$$

- $P(T \mid S)$ learned from aligned data.
- Often implemented using noisy-channel model or direct conditional model.

4. Word Alignment (Idea in MT)

- Find which source words correspond to which target words.
- Alignment is a set of links between words.
- Learned from parallel corpora using probabilistic models (e.g., IBM models).
- Helps estimate translation probabilities $P(\text{target} \mid \text{source})$.

Example: Word Alignment



5. Information Retrieval (IR)

- Goal: rank documents by relevance to a query.
- Probabilistic model: rank by $P(\text{relevant} \mid \text{query, document})$.
- Query and documents represented as bags of words (e.g., tf-idf, language models).
- Rank documents by decreasing probability of relevance.

IR as probabilistic inference:

$$P(\text{relevant} \mid q, d) \propto P(q \mid d)P(d)$$

- Widely used in web search engines.
- Extensions include relevance feedback, query expansion, and learning to rank.

6. Information Extraction (IE)

- Goal: automatically extract structured information (entities, relations, events) from text.
- Uses probabilistic sequence labeling or structured models (e.g., HMM, CRF).
- Common tasks:
  - Named entity recognition (person, organization, location, date).
  - Relation extraction (e.g., works-for).
  - Event extraction (e.g., product launch).
- Handles uncertainty and ambiguity in natural language.

Example: Named Entity Recognition

Barack	Obama	visited	Paris	in	2014	.
PER	PER	O	LOC	O	DATE	O

7. Word-Sense Disambiguation (WSD)

- Words often have multiple possible senses.
- Goal: find the most likely sense given context.
- Use probabilistic models (e.g., Naive Bayes, language models, supervised classifiers).
- Features: surrounding words, part of speech, syntactic relations, etc.

Example: Word Sense Disambiguation

(a) He banked the plane to the left.

bank (river side) ✗ bank (turn) ✓

(b) She went to the bank to deposit money.

bank (river side) ✗ bank (financial) ✓

8. Applications of Probabilistic Language Processing

Application	Goal	Probabilistic Model / Idea	Example
Speech recognition	Transcribe speech to text	$P(\text{signal} \mid W)P(W)$	Dictation, voice assistants
Machine translation	Translate text between languages	$P(T \mid S)$	English → French
Information retrieval	Rank relevant documents	$P(\text{relevant} \mid q, d)$	Web search
Information extraction	Extract entities, relations, events	Sequence labeling, structured models	Find people, places, companies
Word-sense disambiguation	Choose correct word sense	$P(\text{sense} \mid \text{context})$	"bank" → financial
Text summarization	Generate concise summary	Probabilistic models of importance	News summarization
Question answering	Find answer in text	$P(\text{answer} \mid \text{question, text})$	Factoid questions
Dialogue systems	Model next utterance	$P(\text{response} \mid \text{context})$	Chatbots, assistants

9. Summary of Key Models

- Language models: $P(W)$ (n-grams, smoothing).
- Acoustic models: $P(\text{signal} \mid W)$ (HMM, deep models).
- Translation models: $P(T \mid S)$ with word alignment.
- Document models: $P(\text{relevant} \mid q, d)$.
- Sequence labeling: $P(\text{labels} \mid \text{words})$ (HMM, CRF).
- Classification models for word senses.

10. Key Takeaways

1. Probabilistic models handle uncertainty in language data.
2. Speech recognition combines acoustic and language models.
3. Machine translation uses parallel corpora and word alignment.
4. Information retrieval ranks documents by relevance probability.
5. Information extraction finds structured information from text.
6. Word-sense disambiguation uses context to choose the correct sense.
7. Probabilistic language processing enables many real-world applications.

Chapter 23 : Probabilistic Language Processing

1. Evaluation Metrics for Language Models

- We need evaluation metrics to measure how well a model predicts or generates language.
- Common metrics: perplexity, accuracy, precision, recall, F1 score.
- Choice of metric depends on the task (e.g., next word prediction, tagging, parsing, machine translation, speech recognition).
- Lower perplexity is better. Higher accuracy, precision, recall and F1 are better.

Key Idea: Evaluation tells us how well the model generalizes to new, unseen data.

2. Perplexity

- Measures how well a probability model predicts a test set.
- Lower perplexity means the model is less surprised and has higher probability for the test data.
- For a test sequence $W = w_1, \ldots, w_N$:

$$PP(W) = 2^{-\frac{1}{N}\sum_{i=1}^{N} \log_2 P(w_i \mid \text{context})}$$

- Equivalent to the inwerse probability of the test set, normalized by length.
- Often reported as word-level perplexity.
- Used for n-gram models, HMMs and other generative models.

3. Accuracy, Precision and Recall

- Accuracy: fraction of correct predictions.

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total predictions}}$$

- Precision: fraction of predicted positives that are correct.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Recall: fraction of actual positives that are recovered.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- F1 Score: harmonic mean of precision and recall.

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

TP: true positive
FP: false positive
FN: false negative

4. Data Sparsity and Its Effects

- In real text, many possible word sequences never occur in the training data (zero counts).
- Data sparsity leads to poor probability estimates and low coverage for unseen events.
- Causes: large vocabulary, limited training data, longer n-grams, complex structures.
- Solutions: smoothing (Laplace, backoff, interpolation), using more data, tying parameters, or using neural models by analogy (not covered in depth here).

Key Idea: Smoothing and backoff help reduce the impact of data sparsity.

5. Common Mistakes and Pitfalls

- Ignoring data sparsity (using raw counts).
- Choosing n too large for available data.
- Not using smoothing (leads to zero probabilities).
- Evaluating on training data instead of a separate test set (overfitting).
- Comparing models using different metrics or different test sets.
- Confusing probability of a sentence with probability of next word.
- Ignoring the effect of tokenization, vocabulary size and unknown words (OOV).
- Relying only on accuracy when other metrics (e.g., F1, perplexity) are more appropriate.

6. Model Comparison

- Different models make different assumptions and are suitable for different tasks.
- Comparison depends on evaluation metric, task and data.
- n-gram: simple, strong baseline, but suffers from sparsity.
- HMM: good for sequential tagging (speech, POS), models hidden states.
- PCFG: good for syntax (parsing), captures hierarchical structure.
- Noisy-channel: good for correction and machine translation, combines model and noisy input.

7. Comparison of Language Models

Model	Main Idea	Strengths	Weaknesses	Typical Tasks	Evaluation Metrics
n-gram model	$P(w_i \mid$ history of n-1 words)	Simple, effective, fast to train	Data sparsity, limited context, no long-range structure	Language modeling, next word prediction	Perplexity, word accuracy
Hidden Markov Model (HMM)	Hidden states generate observable words	Good for sequential tasks, handles uncertainty	Limited context, independence assumptions	POS tagging, speech recognition	Accuracy, F1, perplexity (on words)
Probabilistic Context-Free Grammar (PCFG)	Generative model for hierarchical syntax structures	Captures syntactic structure, handles disambiguation	Simplifying assumptions, less effective for long-distance dependencies	Parsing, syntax analysis	Parse accuracy, F1 (constituents), likelihood / perplexity
Noisy-Channel Model	Choose most likely intended sentence given noisy input	Effective for correction and translation, handles noisy input	Needs good models for $P(w)$ and $P(\text{noisy} \mid w)$, can be computationally costly	Spelling correction, machine translation, ASR error correction	Word / sentence accuracy, BLEU (perplexity for LM)

8. Likely Exam Questions

1. Define perplexity. Derive the formula and explain its interpretation.
2. How do accuracy, precision, recall and F1 differ? When is each used?
3. Explain the problem of data sparsity and how smoothing helps.
4. Compare n-gram models, HMMs, PCFGs and noisy-channel models. Discuss their assumptions, strengths and weaknesses.
5. Given counts or probabilities, compute perplexity, precision, recall, or F1.
6. Short notes on: smoothing, backoff, interpolation, evaluation metrics for parsing or tagging.

9. Study Tips

- Understand the meaning and intuition behind each evaluation metric.
- Work through numeric examples (compute perplexity, precision, recall, F1).
- Be clear about the assumptions and limitations of each model (n-gram, HMM, PCFG, noisy-channel).
- Practice comparing models for different tasks.
- Revise how data sparsity arises and the role of smoothing / backoff.
- Focus on how to choose appropriate metrics for different applications.
- Look at past exam questions and practice short and long answers.

10. Key Takeaways

1. Evaluation metrics measure how well a model predicts or generates language.
2. Lower perplexity is better; higher accuracy, precision, recall and F1 are better.
3. Data sparsity is a fundamentad challenge in language modeling.
4. Smoothing, backoff and interpolation reduce the impact of sparse data.
5. n-gram, HMM, PCFG and noisy-channel models have different assumptions and strengths.
6. Choose evaluation metrics appropriate for the task.
7. Careful evaluation and comparison are essential for selecting the best model.